# COS 30049 Computing Technology Innovation Project

## Week7: Clustering and Classification Tasks in Python

Semester 2

# Acknowledgement of Country

We respectfully acknowledge the Wurundjeri People of the Kulin Nation, who are the Traditional Owners of the land on which Swinburne's Australian campuses are located in Melbourne's east and outer-east, and pay our respect to their Elders past, present and emerging.

We are honoured to recognise our connection to Wurundjeri Country, history, culture, and spirituality through these locations, and strive to ensure that we operate in a manner that respects and honours the Elders and Ancestors of these lands.

We also respectfully acknowledge Swinburne's Aboriginal and Torres Strait Islander staff, students, alumni, partners and visitors.

We also acknowledge and respect the Traditional Owners of lands across Australia, their Elders, Ancestors, cultures, and heritage, and recognise the continuing sovereignties of all Aboriginal and Torres Strait Islander Nations.



# Objectives:

Introduction to DBSCAN implementation
Practical steps in Logistic Regression
Practical steps in KNN
Code evaluation and visualization techniques

SWINBURNE UNIVERSITY OF TECHNOLOGY

# Clustering - DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is an unsupervised clustering algorithm. It identifies clusters in data by finding regions of high density separated by regions of low density. Unlike k-means, it doesn't require the number of clusters to be specified beforehand.

**Key Characteristics of DBSCAN:**

Handles Arbitrary Shapes: DBSCAN can find clusters of any shape, which makes it powerful for datasets where the clusters are not spherical (unlike k-means).

Noise Handling: It classifies points that don't belong to any cluster as noise, useful in datasets with outliers.

**When to Use DBSCAN:**

Use DBSCAN when you don't know the number of clusters in advance.

It's effective in datasets with non-spherical clusters and noise.

Ideal for data where the clusters are densely packed and separated by sparse regions.

SWINBURNE UNIVERSITY OF TECHNOLOGY

# Clustering - DBSCAN

## Prepare data

### 1. Generating Data using make_moons:

The make_moons function from sklearn.datasets generates synthetic data with two interleaving half circles. It is often used for testing clustering algorithms like DBSCAN because the clusters are not linearly separable, making them a great example for demonstrating how DBSCAN identifies arbitrary-shaped clusters.

```
from sklearn.datasets import make_moons

# Generate a synthetic dataset with two interleaving half circles
X, y = make_moons(n_samples=500, noise=0.1, random_state=42)

# Plot the dataset
plt.scatter(X[:, 0], X[:, 1], c=y, cmap='viridis', s=50)
plt.title('Original Data')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```

- n_samples=500: The total number of data points generated.
- noise=0.1: Controls the level of noise, or deviation, from the ideal half-circle shapes.
- random_state=42: Ensures reproducibility by fixing the random number generation.

SWINBURNE UNIVERSITY OF TECHNOLOGY

# Clustering - DBSCAN

## Prepare data

### 2. Why Use StandardScaler for Normalization?

DBSCAN is sensitive to the scale of the data. If one feature has a much larger scale than another, the algorithm will treat the larger-scale feature as more important. To avoid this, we normalize the data so that all features have the same scale. The StandardScaler from sklearn.preprocessing transforms the data by removing the mean and scaling it to unit variance, ensuring all features contribute equally to the distance calculation in DBSCAN.

```
from sklearn.preprocessing import StandardScaler

# Normalize the dataset
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

- fit_transform(X): The method first calculates the mean and standard deviation of the data (fit) and then scales the data (transform) based on these values.
- Normalizing ensures that features with larger ranges don’t dominate the clustering process. This is particularly important when features are on different scales, like in many real-world datasets.

SWINBURNE UNIVERSITY OF TECHNOLOGY

# Clustering - DBSCAN

## Prepare the data

### 3. Visualizing the Normalized Data:

•After scaling, the data is plotted to confirm the transformation. This step is critical to show students how the data looks after normalization, which is a common preprocessing task in clustering.

```
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=y, cmap='viridis', s=50)
plt.title('Normalized Data')
plt.xlabel('Feature 1 (Scaled)')
plt.ylabel('Feature 2 (Scaled)')
plt.show()
```

SWINBURNE UNIVERSITY OF TECHNOLOGY

# Clustering - DBSCAN

## DBSCAN Implementation

### 3. Key Parameters in DBSCAN:

***eps (Epsilon):*** Defines the maximum distance between two points for them to be considered as part of the same neighborhood. A smaller eps results in more clusters, while a larger eps may result in fewer clusters.

***min_samples:*** The minimum number of points required to form a dense region (i.e., a cluster). This includes the point itself and its neighbors. If a point has fewer than min_samples points within eps distance, it is classified as noise or a border point.

```
from sklearn.cluster import DBSCAN

# Apply DBSCAN algorithm
dbscan = DBSCAN(eps=0.3, min_samples=5)
dbscan.fit(X_scaled)

# Extract cluster labels
labels = dbscan.labels_
```

***DBSCAN(eps=0.3, min_samples=5):*** The model is initialized with an eps of 0.3 and min_samples of 5. This means any point that has at least 5 neighbors within 0.3 units of distance will form a cluster.

***labels = dbscan.labels_:*** The .labels_ attribute contains the cluster assignments for each data point. If a point is labeled as -1, it is considered noise.

SWINBURNE UNIVERSITY OF TECHNOLOGY

# Clustering - DBSCAN

## DBSCAN Implementation

### Impact of Parameter Choices:

•**Choosing ε (eps)**: A smaller value of eps will result in more clusters, but might also treat points in dense areas as noise. A larger eps value may create fewer, larger clusters, or may even merge clusters together.

•**Choosing min_samples**: A smaller min_samples value could lead to more, smaller clusters, including small groups of points being identified as clusters. Larger min_samples values ensure that only significant groups of points are considered clusters.

**It is often recommended to experiment with different values for eps and min_samples to achieve the best clustering performance for a specific dataset.**

Also, you can use some tools/methos to choose them.

SWIN BUR NE SWINBURNE UNIVERSITY OF TECHNOLOGY

# Clustering - DBSCAN

## Practical Considerations for DBSCAN Parameter Selection

❖ K-Distance Graph Example:

- The k-distance graph helps visualize the optimal value for ε. Below the elbow point, data points are close to each other and belong to a dense region; above the elbow, points are more distant and sparse.
- Plotting k-distance graph:

```python
from sklearn.neighbors import NearestNeighbors
import numpy as np
import matplotlib.pyplot as plt

# Fit NearestNeighbors model
neigh = NearestNeighbors(n_neighbors=4)
neigh.fit(data)
distances, indices = neigh.kneighbors(data)

# Sort distances
distances = np.sort(distances[:, 3], axis=0)
plt.plot(distances)
plt.ylabel('k-distance')
plt.xlabel('Data Points')
plt.title('K-Distance Graph')
plt.grid(True)
plt.show()
```

SWINBURNE UNIVERSITY OF TECHNOLOGY

# Clustering - DBSCAN

## Visualization of DBSCAN Results

- Visualization: After DBSCAN has clustered the data, the results are visualized using a scatter plot where each point is colored according to its cluster label. Noise points are usually assigned a separate color (typically labeled as -1).
- The code below uses matplotlib to generate a scatter plot of the clusters. Each cluster is represented by a different color, and noise points are marked separately.

```
import matplotlib.pyplot as plt
import numpy as np

# Plot the clusters
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=labels, cmap='plasma', s=50)
plt.title('DBSCAN Clusters')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```

SWINBURNE UNIVERSITY OF TECHNOLOGY

# Clustering - DBSCAN

## Visualization of DBSCAN Results

```
import matplotlib.pyplot as plt
import numpy as np

# Plot the clusters
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=labels, cmap='plasma', s=50)
plt.title('DBSCAN Clusters')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```

**plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=labels, cmap='plasma', s=50)**: The data points are plotted on a 2D scatter plot, colored by their cluster labels (c=labels). Points assigned the same label (cluster) will have the same color.

**cmap='plasma'**: The colormap used to display different clusters. You can change the colormap depending on the preference.Noise points (labeled -1) will have a different color, indicating that DBSCAN has identified them as outliers.

SWINBURNE UNIVERSITY OF TECHNOLOGY

# Clustering - DBSCAN

## DBSCAN Evaluation

**•Silhouette Score as a Metric for Clustering:**

The **Silhouette Score** is a popular metric for evaluating clustering performance. It measures how similar a point is to its own cluster compared to other clusters. The score ranges from -1 to 1:

- **1**: The point is perfectly clustered.
- **0**: The point lies on the boundary between clusters.
- **-1**: The point is misclassified.

We use the silhouette_score function from sklearn.metrics to compute the score.

```
from sklearn.metrics import silhouette_score

# Calculate the silhouette score
silhouette_avg = silhouette_score(X_scaled, labels)
print(f'Silhouette Score: {silhouette_avg:.2f}')
```

SWINBURNE UNIVERSITY OF TECHNOLOGY

# Clustering - DBSCAN

**DBSCAN Evaluation**

```
from sklearn.metrics import silhouette_score

# Calculate the silhouette score
silhouette_avg = silhouette_score(X_scaled, labels)
print(f'Silhouette Score: {silhouette_avg:.2f}')
```

- **silhouette_score(X_scaled, labels)**: This function computes the silhouette score based on the scaled dataset and the labels assigned by DBSCAN.
- **silhouette_avg**: The average silhouette score for all the points in the dataset. A higher score indicates better-defined clusters.

SWINBURNE UNIVERSITY OF TECHNOLOGY

# Clustering - DBSCAN

## DBSCAN Evaluation

### Interpreting the Silhouette Score

- A high silhouette score (close to 1) indicates that the clusters are well-separated and the points are well-grouped within their respective clusters.
- A low or negative silhouette score suggests that the clusters are poorly defined, with points either on cluster boundaries or misclassified as part of the wrong cluster.
- **Important Discussion Points:**

  •The silhouette score is particularly useful when you are trying different values of eps and min_samples in DBSCAN, as it provides a quantitative way to evaluate how well the clusters are formed.

  •However, the silhouette score can be biased when clusters vary significantly in size or density, so it should be used along with visual analysis of the cluster shapes.

SWINBURNE UNIVERSITY OF TECHNOLOGY

# Clustering - DBSCAN

- Effect of Noise and Outliers:
  - DBSCAN is particularly useful in datasets with noise and outliers. Choosing appropriate ε and MinPts values ensures that noise points are excluded from clusters.
- Dataset Characteristics:
  - The shape and density of clusters in the data can affect parameter selection. For example, if clusters vary significantly in size or density, it may be difficult to choose a single ε value that works well across the entire dataset.
- Parameter Sensitivity:
  - DBSCAN can be sensitive to the choice of parameters, especially when clusters have varying densities. In such cases, alternative density-based clustering methods like OPTICS may perform better.

SWINBURNE UNIVERSITY OF TECHNOLOGY

# Classification - Logistic Regression

## ❖ Data Preparation

Using the load_breast_cancer Dataset:

The load_breast_cancer dataset from sklearn.datasets is a built-in dataset for binary classification. It contains features related to breast cancer diagnosis (malignant or benign tumors).

```
from sklearn.datasets import load_breast_cancer

# Load breast cancer dataset
data = load_breast_cancer()
X = data.data  # Features
y = data.target  # Labels (0: Malignant, 1: Benign)
```

X = data.data: The feature matrix with 30 features representing different characteristics of the cell nuclei.

y = data.target: The binary target labels (0 = malignant, 1 = benign).

SWINBURNE UNIVERSITY OF TECHNOLOGY

# Classification - Logistic Regression

## ❖ Data Preparation

**•Dataset Splitting:**

We split the dataset into **training** and **test** sets using train_test_split to evaluate the model's performance on unseen data.

```
# Split the data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

**•test_size=0.2**: 20% of the data is reserved for testing.

**•random_state=42**: Ensures reproducibility by fixing the randomness of the data splitting.

SWINBURNE UNIVERSITY OF TECHNOLOGY

# Classification - Logistic Regression

## ❖ Data Preparation

•Standardization with StandardScaler:

Standardization is critical for logistic regression since it assumes the input features are on a similar scale.

This improves the convergence of the algorithm and helps it make better predictions.

```
from sklearn.preprocessing import StandardScaler

# Standardize the dataset (mean=0, variance=1)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

•**fit_transform(X_train):** Calculates the mean and standard deviation from the training set and scales the data.

•**transform(X_test):** Uses the same scaling parameters from the training set to transform the test set, ensuring no data leakage.

SWINBURNE UNIVERSITY OF TECHNOLOGY

# Classification - Logistic Regression

## ❖ Logistic Regression Model Setup

Logistic Regression Overview:

Logistic regression is a supervised learning algorithm used for binary classification. It estimates the probability that a given input belongs to a particular class by using a logistic (sigmoid) function.

Model Instantiation:

The LogisticRegression() function from sklearn.linear_model is used to create the logistic regression model.

```
from sklearn.linear_model import LogisticRegression

# Instantiate the Logistic Regression model
log_reg = LogisticRegression(random_state=42)
```

- **random_state=42**: Ensures the same results are obtained each time the code is run.

SWINBURNE UNIVERSITY OF TECHNOLOGY

# Classification - Logistic Regression

❖ **Logistic Regression Model Setup**

Key Parameters:

- penalty: Specifies the type of regularization to use. Regularization helps prevent overfitting. The default is 'l2', which refers to ridge regularization.
- C: Inverse of regularization strength. Smaller values specify stronger regularization. Default is 1.0.
- solver: The algorithm used for optimization. Defaults to 'lbfgs', which is appropriate for small datasets. Other solvers can be used for larger datasets or different penalty types.

SWINBURNE UNIVERSITY OF TECHNOLOGY

# Classification - Logistic Regression

## ❖ Logistic Regression: Model Fitting

**How .fit() Works:**

•The .fit() function trains the logistic regression model by finding the optimal parameters (weights and bias) that minimize the cost function, typically the log-loss function for classification.

**Code Walkthrough:**

•The code below fits the logistic regression model to the scaled training data

```
# Fit the Logistic Regression model
log_reg.fit(X_train_scaled, y_train)
```

**log_reg.fit(X_train_scaled, y_train)**: This trains the model using the scaled training data (X_train_scaled) and the corresponding labels (y_train).

SWINBURNE UNIVERSITY OF TECHNOLOGY

# Classification - Logistic Regression

## ❖ Logistic Regression: Model Fitting

- What Happens Under the Hood:
  - Logistic regression uses gradient descent to minimize the cost function (log loss). The optimization process updates the model's parameters (weights and bias) until convergence or until a stopping criterion is met.
  - Regularization (if used) is applied during this process to penalize large coefficients, helping to prevent overfitting.
- Important Point to Discuss:
  - The .fit() function is where the model learns from the training data. The algorithm iterates over the data multiple times, adjusting weights based on the error between predictions and actual labels.

SWINBURNE UNIVERSITY OF TECHNOLOGY

# Classification - Logistic Regression

❖ **Logistic Regression: Prediction**

Prediction with .predict():

After training the model, we use the .predict() function to make predictions on the test data. The logistic regression model outputs binary class labels (0 or 1) based on the learned weights and the sigmoid function.

```
# Make predictions on the test data
y_pred = log_reg.predict(X_test_scaled)
```

**y_pred = log_reg.predict(X_test_scaled)**: This function uses the trained model to predict the labels for the test data (X_test_scaled).

SWINBURNE UNIVERSITY OF TECHNOLOGY

# Classification - Logistic Regression

## ❖ Logistic Regression: Prediction

What the Output Represents:

The .predict() method outputs binary class labels:

1: Predicted as benign (non-cancerous).

0: Predicted as malignant (cancerous).

Internally, the model calculates the probability that each data point belongs to class 1 (benign) using the logistic (sigmoid) function. If the probability is greater than 0.5, the model assigns the label 1; otherwise, it assigns the label 0.

**Optional Extension:**

•To show how the model calculates probabilities, you can use the .predict_proba() method.

```
# Predict probabilities for each class
y_proba = log_reg.predict_proba(X_test_scaled)
```

**predict_proba()** returns the probability of each class (class 0 and class 1). For each sample, it provides the probability of being in each class, which can give students deeper insight into how confident the model is about its predictions.

SWINBURNE UNIVERSITY OF TECHNOLOGY

# Classification - Logistic Regression

## ❖ Logistic Regression: Prediction

What the Output Represents:

The .predict() method outputs binary class labels:

1: Predicted as benign (non-cancerous).

0: Predicted as malignant (cancerous).

Internally, the model calculates the probability that each data point belongs to class 1 (benign) using the logistic (sigmoid) function. If the probability is greater than 0.5, the model assigns the label 1; otherwise, it assigns the label 0.

**Optional Extension:**

•To show how the model calculates probabilities, you can use the .predict_proba() method.

```
# Predict probabilities for each class
y_proba = log_reg.predict_proba(X_test_scaled)
```

**predict_proba()** returns the probability of each class (class 0 and class 1). For each sample, it provides the probability of being in each class, which can give students deeper insight into how confident the model is about its predictions.

SWINBURNE UNIVERSITY OF TECHNOLOGY

# Classification

## ❖ KNN: Data Preparation

- Revisiting the Same Dataset:
  - For consistency, we will continue to use the load_breast_cancer dataset, which has been preprocessed and split into training and test sets.
  - Why Use the Same Dataset? It allows for a direct comparison between Logistic Regression and KNN models in terms of performance and behavior, as both algorithms will be trained on the same data.
- KNN Also Uses Standardized Data:
  - Why Standardization is Important for KNN:

    KNN is a distance-based algorithm, which means it calculates the distance between data points to determine their "neighbors." If the data is not standardized, features with larger ranges will dominate the distance calculation, leading to biased predictions.

The dataset has already been scaled using StandardScaler earlier. We can directly use the standardized training and test sets (X_train_scaled and X_test_scaled).

```
# X_train_scaled and X_test_scaled are already standardized
print(f"Number of training samples: {X_train_scaled.shape[0]}")
print(f"Number of test samples: {X_test_scaled.shape[0]}")
```

SWINBURNE UNIVERSITY OF TECHNOLOGY

# Classification

## ❖ KNN Model Setup

- **K-Nearest Neighbors Overview:**
  - KNN is a simple, **instance-based learning** algorithm. It classifies a data point based on the majority class of its nearest neighbors in the feature space.
  - The primary hyperparameter is **n_neighbors**, which controls how many neighbors are considered when making predictions.
- **Code Breakdown for KNN Model:**
  - The KNeighborsClassifier() function from sklearn.neighbors is used to create the KNN model.

```
from sklearn.neighbors import KNeighborsClassifier

# Instantiate the KNN model with 5 neighbors
knn = KNeighborsClassifier(n_neighbors=5)
```

**n_neighbors=5**: This specifies that the algorithm will consider the **5 nearest neighbors** to make its prediction. For example, if 3 out of 5 neighbors belong to class 1, the new data point will be classified as class 1.

SWINBURNE UNIVERSITY OF TECHNOLOGY

# Classification

## ❖ KNN Model Setup

### ➢ Key Parameter: n_neighbors:

- What Does n_neighbors Control?
  - It defines how many neighbors should be considered when determining the class of a data point.
  - Small values (e.g., 1 or 3) lead to models that are more sensitive to noise but can capture local patterns better.
  - Larger values smooth out predictions and reduce the impact of noisy points, but they can make the model less sensitive to finer details.
- Guideline for Choosing n_neighbors:
  - Start with an odd number (e.g., 3, 5) to avoid ties in binary classification.
  - Use cross-validation to experiment with different values and choose the one that gives the best results.

SWIN BUR NE SWINBURNE UNIVERSITY OF TECHNOLOGY

# Classification

## ❖KNN: Model Fitting and Prediction

•**Fitting the KNN Model with .fit():**Similar to Logistic Regression, we use the .fit() method to train the KNN model. However, unlike Logistic Regression, KNN does not "learn" weights during training. Instead, it simply stores the training data and computes distances at prediction time.

```
# Fit the KNN model
knn.fit(X_train_scaled, y_train)
```

- **knn.fit(X_train_scaled, y_train)**: This function stores the scaled training data (X_train_scaled) and corresponding labels (y_train). No learning happens at this stage—KNN simply saves the data for later use.

SWINBURNE UNIVERSITY OF TECHNOLOGY

# Classification

## ❖KNN: Model Fitting and Prediction

•**Making Predictions with .predict():**After fitting the model, we use the .predict() method to classify new (test) data points based on the nearest neighbors in the training set.

```
# Predict on the test data
y_pred_knn = knn.predict(X_test_scaled)
```

- **knn.predict(X_test_scaled**): For each point in the test set, the algorithm calculates the distance to every point in the training set. It then finds the n_neighbors closest points and assigns the majority class among those neighbors as the prediction.

SWIN BUR NE SWINBURNE UNIVERSITY OF TECHNOLOGY

# Classification

## ❖Accuracy and Precision Evaluation

**•Overview of Evaluation Metrics:**

•For classification tasks, it is important to evaluate how well the model performs. Two commonly used metrics are accuracy and precision. These metrics help understand how the model is making predictions.

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}} = \frac{TP + TN}{TP + TN + FP + FN}$$

```python
from sklearn.metrics import accuracy_score

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")
```

**accuracy_score(y_test, y_pred)**: This function calculates the accuracy by comparing the true labels (y_test) and the predicted labels (y_pred).

SWINBURNE UNIVERSITY OF TECHNOLOGY

# Classification

## ❖Accuracy and Precision Evaluation

**•Overview of Evaluation Metrics:**

•For classification tasks, it is important to evaluate how well the model performs. Two commonly used metrics are accuracy and precision. These metrics help understand how the model is making predictions.

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}} = \frac{TP}{TP + FP}$$

```
from sklearn.metrics import precision_score

# Calculate precision
precision = precision_score(y_test, y_pred)
print(f"Precision: {precision:.2f}")
```

**precision_score(y_test, y_pred):** This function calculates the precision score by comparing the true labels (y_test) and predicted labels (y_pred).

SWINBURNE UNIVERSITY OF TECHNOLOGY

# Classification

## ❖Recall and F1-Score Evaluation

Recall (also known as sensitivity or true positive rate) is the ratio of correctly predicted positive instances to all actual positive instances. It answers the question: Of all the actual positive cases, how many did we correctly predict?

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}} = \frac{TP}{TP + FN}$$

```python
from sklearn.metrics import recall_score

# Calculate recall
recall = recall_score(y_test, y_pred)
print(f"Recall: {recall:.2f}")
```

**recall_score(y_test, y_pred)**: This function calculates the recall by comparing the true labels (y_test) and predicted labels (y_pred).

SWINBURNE UNIVERSITY OF TECHNOLOGY

# Classification

You can use accuracy to evaluate classification tasks, but sometimes accuracy may not objectively reflect the actual performance of the model.

Jaccard Score and F1 Score are two common evaluation metrics used to assess the performance of classification models.

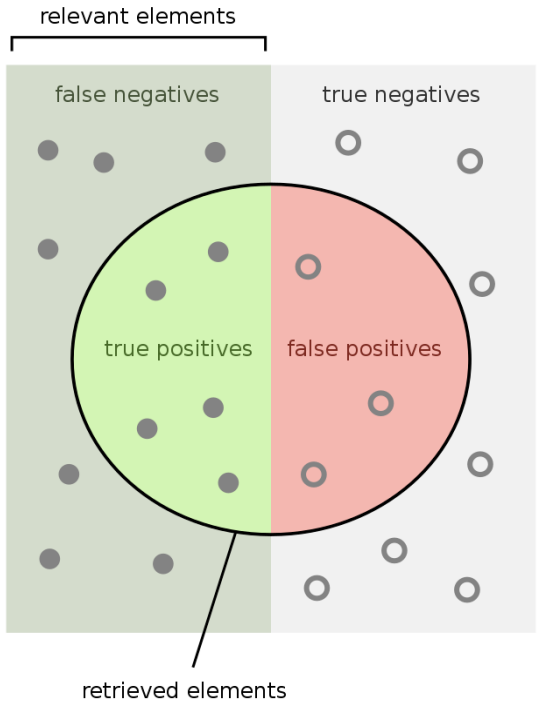
**Jaccard Score (Jaccard Index)**

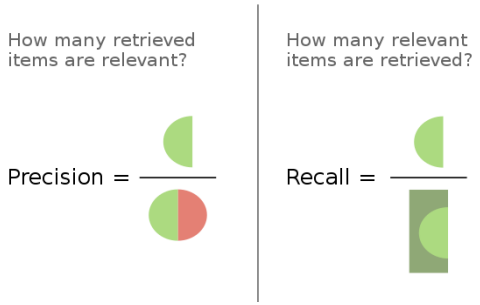
The Jaccard Score, also known as the Jaccard Index or Intersection over Union (IoU), measures the similarity between two sets. For classification problems, it is typically used to evaluate the overlap between predicted labels and true labels.

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

SWIN BUR NE SWINBURNE UNIVERSITY OF TECHNOLOGY

# Classification




•F1-score is the harmonic mean of precision and recall. It is useful when there is an uneven class distribution or when both precision and recall are important.

$$F_1 = \frac{2}{\mathrm{recall}^{-1} + \mathrm{precision}^{-1}} = 2\frac{\mathrm{precision} \cdot \mathrm{recall}}{\mathrm{precision} + \mathrm{recall}} = \frac{2\mathrm{tp}}{2\mathrm{tp} + \mathrm{fp} + \mathrm{fn}}.$$

```python
from sklearn.metrics import f1_score

# Calculate F1-Score
f1 = f1_score(y_test, y_pred)
print(f"F1-Score: {f1:.2f}")
```

**f1_score(y_test, y_pred)**: This function calculates the F1-score by considering both precision and recall.

SWINBURNE UNIVERSITY OF TECHNOLOGY

# Thanks

SWINBURNE UNIVERSITY OF TECHNOLOGY